

كرة احترافي — Data Science

علم البيانات

Tools for Data Science — من الأساسيات إلى بيئات العمل السحابية

Machine & Deep Learning

مكتبات Python

إدارة الأنكود والبيانات

بيئات التطوير

أدوات سحابية

فهرس المحتويات

Course Overview	1. نظرة عامة على المقرر
Task Categories	2. فئات مهام علم البيانات
Tools Categories	3. فئات الأدوات (بالتفصيل مع أمثلة)
Open-Source	4. الأدوات مفتوحة المصدر
Commercial	5. الأدوات التجارية
Cloud-Based	6. الأدوات السحابية
Key Takeaways	7. خلاصة الوحدة الأولى
Python Libraries	8. مكتبات Python لعلم البيانات
APIs	9. واجهات البرمجة APIs
Datasets & CDLA	10. مجموعات البيانات والترخيص
IBM DAX	11. مستودع IBM لأصول البيانات DAX
ML Models	12. نماذج تعلم الآلة
Deep Learning Tools	13. أدوات التعلم العميق
Key Takeaways	14. خلاصة الوحدة الثانية
From Slides	15. إضافات من صور المحاضرة (Session 3)
Review	16. جدول المقارنة الشامل + أسئلة مراجعة

الوحدة الأولى — Tools for Data Science: الأساسيات

1. نظرة عامة على المقرر (Course Overview)

المقرر يغطي الأدوات والبيئات الأساسية التي يحتاجها في علم البيانات، من مكتبات وباكديجات ومجموعات بيانات لازمة للتعامل مع ال Machine Learning وال Big Data. هتشتغل بلغات زي Python و R و SQL، وهتستخدم أدوات زي Jupyter Notebooks و RStudio و GitHub للكّابة والتعاون وإدارة المشاريع.

ليه الموضوع ده مهم؟ لأن ال Data Scientist مش بس بيكتب كود، لازم يعرف يختار الأداة الصح لكل مرحلة من مراحل المشروع (جمع بيانات → تنظيف → بناء موديل → نشر → مراقبة)، وده اللي هيوضحه الملخص ده.

2. فئات مهام علم البيانات (Data Science Task Categories)

المهمة	الوصف
Data Management	تجميع وتخزين واسترجاع البيانات بكفاءة من مصادر زي السوشيال ميديا أو أجهزة الاستشعار (sensors).
Data Integration & Transformation (ETL)	استخراج البيانات من مصادر مختلفة، تحويل شكلها، وتحميلها في مخازن بيانات (data warehouses).
Data Visualization	استخدام أدوات لعمل تمثيل بصري (رسومات، خرائط) عشان توضح الرؤى المستخرجة من البيانات.
Model Building	تدريب نماذج تعلم آلة لاكتشاف الأنماط (patterns) في البيانات.
Model Deployment	نشر النماذج في بيئات إنتاج (production) عشان نتاخذ قرارات مبنية على البيانات.
Model Monitoring	التأكد من دقة النموذج وعدالته (fairness) وقوته (robustness) بشكل مستمر بعد النشر.

لاحظ: المهام دي مرتبة بالترتيب المنطقي اللي يحصل بيه أي مشروع Data Science فعلياً — من جمع البيانات لحد مراقبة النموذج بعد ما يشتغل في الإنتاج.

3. فئات الأدوات (Tools Categories) — بالتفصيل

دي أهم نقطة في الملف، وهي تقسيم أي أداة في عالم ال Data Science تحت 5 فئات رئيسية:

3.1 إدارة أصول الكود — Code Asset Management

أدوات زي **Git** و**GitHub** تستخدم للتحكم في إصدارات الكود (version control)، ويتسمح لأكثر من شخص يشتغلوا على نفس الكود بالتعاون من غير ما يضيعوا شغل بعض.

مثال: فريق يبني موديل تصنيف صور — كل عضو يشتغل على branch منفصل في GitHub، ولما الكود يخلص يتم عمل merge للنسخة الرئيسية (main)، مع إمكانية الرجوع لأي نسخة قديمة لو حصلت مشكلة.

3.2 إدارة أصول البيانات — Data Asset Management

مش بس الكود بيتغير في مشروع Data Science، البيانات نفسها بتغير باستمرار (إضافة صفوف، تنظيف، feature engineering، أخذ عينات). عشان كده محتاجين أدوات "Version Control" لكن للبيانات مش للكود.

الأداة	وظيفتها
DVC (Data Version Control)	زي Git بالظبط لكن متخصص في تتبع نسخ البيانات والموديلات الكبيرة، ويحفظ الفروقات (diffs) مش الملف كامل عشان يوفر مساحة.
MLflow	بيتبع تجارب التدريب (experiments) وال parameters المستخدمة، ومرتبطة بمرحلة ال Monitoring كإن.
Git LFS (Large File Storage)	إضافة ل Git بتسمح له يتعامل بكفاءة مع ملفات كبيرة زي الداتاسيتس والصور.

الخلاصة: Code Asset Management (Git/GitHub) يتبع تغييرات الكود، بينما Data Asset Management (DVC وأمثاله) يتبع تغييرات البيانات نفسها — والاثنين مع بعض بضممنوا إنك تقدر "تعيد إنتاج" (reproduce) أي نتيجة: نفس الكود + نفس نسخة البيانات = نفس النتيجة بالظبط.

3.3 بيئات التطوير — Development Environments

دي البرامج اللي بتكتب وتجرب فيها الكود. أشهرها: **Jupyter Notebooks**، **RStudio**، **Apache Zeppelin**، بالإضافة ل **Google Colab**، **VS Code**، **Anaconda**، **PyCharm**. الجدول التالي يوضح الفرق العملي بينهم:

الأداة	نوعها	مميزاتها
Jupyter Notebook	بيئة تفاعلية (Cell-based)	الكود بيتشغل في خلايا منفصلة، مثالية لتحليل البيانات لأنك بتشوف نتيجة كل خطوة فوراً.
Google Colab	نسخة سخاية من Jupyter	تشغل من المتصفح من غير تنصيب، وتديك GPU/TPU مجاناً — مفيدة جداً لـ Deep Learning.
Anaconda	توزيع (Distribution) وليس IDE بمفرده	تجمع Python + مكتبات + أداة إدارة بيئات (conda)، ويبيجي معاه Jupyter و Spyder. بتستخدمه عشان تنظم البيئات وتجنب تعارض المكتبات.

VS Code	محرر أكواد عام (Code Editor)	مرن جداً ويدعم لغات كثير، ويتقدر تضيفه extension للـ Jupyter نفسه.
PyCharm	IDE كامل مخصص لـ Python	أقوى في الـ debugging والمشاريع الكبيرة، لكن أثقل شوية من VS Code.

خلاصة سريعة: Colab و Jupyter يتميزوا في التحليل التفاعلي السريع (data exploration)، أما VS Code و PyCharm فيتميزوا في بناء مشاريع برمجية متكاملة (production code)، و Anaconda هو اللي بيدبر البيئة كلها من غير ما يحصل تعارض بين المكتبات.

3.4 بيئات التنفيذ — Execution Environments

دي المنصات اللي بتوفرلك قوة حوسبة (compute power) لتشغيل وتنفيذ الكود من غير ما تحتاج جهاز قوي عندك، خصوصاً لما تكون شغال على بيانات ضخمة أو تدريب موديلات تحتاج GPU/TPU. من أمثلتها أدوات مفتوحة المصدر زي **Apache Spark** و **Apache Flink**، وكان المنصات السحابية الكبرى:

Google Cloud	IBM Cloud	AWS (Amazon Web Services)
--------------	-----------	---------------------------

مثال عملي: عايز تدرب موديل Deep Learning محتاج GPU قوية، بدل ما تشتري كارت شاشة غالي، بتفتح instance على Google Cloud أو AWS، تشغل عليه الكود، تدفع بس على وقت الاستخدام، وبعد ما تخلص تقفله.

3.5 الأدوات المتكاملة بصرياً — Fully Integrated Visual Tools

دي أدوات "شاملة" بتغطي كل مراحل الـ Data Science Workflow في مكان واحد بدل ما تستخدم أداة منفصلة لكل مرحلة. أشهر مثال: **IBM Watson Studio**.

الأداة	التركيز
IBM Watson Studio	بيئة كاملة تقدر فيها تدبر بياناتك، تبني الموديل، تدربه، تنشره (deploy)، وتراقبه — كله من واجهة واحدة، ويدعم Jupyter و Python و R جواه.
IBM Cognos Analytics	أقرب لأداة Business Intelligence (BI)، تركيزها الأساسي على التقارير والـ Dashboards والـ visualization لدعم القرارات الإدارية، مش بالضرورة بناء موديلات ML زي Watson Studio.

الفكرة الأساسية: الأدوات دي بتوفر عليك إنك تنقل بياناتك بين أكثر من برنامج منفصل — وده اللي بيتقال عليه "Fully Integrated".

4. الأدوات مفتوحة المصدر (Open-Source Tools)

الفئة	أمثلة
Data Management	MySQL, PostgreSQL, MongoDB, Hadoop

Apache AirFlow, SparkSQL — لنقل ونقل البيانات بين الأنظمة	Data Integration
PixieDust, Kibana, Apache Superset	Data Visualization
TensorFlow, Kubernetes, Seldon — لبناء ونشر ومراقبة النماذج	Model Building
IBM AI Fairness 360 — يتأكد إن النماذج دقيقة وعادلة وقابلة للتفسير	Monitoring

5. الأدوات التجارية (Commercial Tools)

أمثلة	الفئة
Oracle Database, Microsoft SQL Server, IBM Db2	Data Management
IBM InfoSphere, Microsoft Integration — لإدارة خطوط أنابيب البيانات المعقدة	Data Integration
Tableau, Microsoft Power BI, IBM Cognos Analytics	Data Visualization
SPSS Modeler, SAS Enterprise Miner (بناء) — SPSS Collaboration (نشر)	Model Building & Deployment

6. الأدوات السحابية (Cloud-Based Tools)

أمثلة	الفئة
Watson Studio, Microsoft Azure Machine Learning	Fully Integrated
AWS DynamoDB, IBM Db2 Cloud	Cloud Data Management
Datameer, IBM Cognos	Cloud Data Visualization
Watson Machine Learning, Amazon SageMaker	Model Building & Deployment

خلاصة الوحدة الأولى (Key Takeaways)

- علم البيانات يشمل إدارة البيانات، بناء النماذج، ونشرها باستخدام مجموعة متنوعة من الأدوات.
- الأدوات مفتوحة المصدر والتجارية مختلفة في الطبيعة، لكن الاثنين أساسيين في شغل Data Science الاحترافي.
- المنصات السحابية بقت مهمة جداً لحلول Data Science القابلة للتوسع (scalable).

الوحدة الثانية — المكتبات، ال APIs، والنماذج

1. مكتبات Python لعلم البيانات

مكتبات الحوسبة العلمية

Pandas أساسية لتنظيف البيانات ومعالجتها، وبتوفر هياكل بيانات قوية زي `DataFrame`.

مكتبات التمثيل البصري

• **Matplotlib**: لعمل رسومات وشارات قابلة للتخصيص.

• **Seaborn**: ممتازة للتصورات المعقدة زي ال `heat maps` وال `violin plots`.

مكتبات تعلم الآلة

• **Scikit-learn**: مكتبة أساسية لمهام ال `regression` وال `classification` وال `clustering`.

• **Keras** (بناء موديلات بسرعة) و **TensorFlow** (تعلم عميق واسع النطاق) لمهام ال `Deep Learning`.

لغات ومكتبات أخرى

Apache Spark يسمح بالحوسبة الموزعة (`cluster computing`)، ويدعم `Python` و `R` و `Scala` و `SQL` لمعالجة البيانات على نطاق واسع.

2. واجهات البرمجة (APIs)

التعريف: ال APIs بتمكن التواصل بين مكونات البرمجيات المختلفة، ويتسمح بالتكامل والتفاعل بين أدوات وخدمات مختلفة.

• أساسيات **REST API**: بتستخدم `HTTP methods` للتعامل مع الطلبات والردود، وعادة بتنقل البيانات بصيغة `JSON`.

• أمثلة **APIs**: `Watson APIs` زي `Text to Speech` و `Language Translator` بتوفر مميزات متقدمة للتفريغ الصوتي والترجمة.

• التواصل بين العميل والسيرفر: ال APIs بتخفي تعقيد ال `backend`، وده يبسهل على المطورين استخدام الخدمات بلغات برمجة مختلفة.

مثال: بدل ما تبني موديل ترجمة من الصفر، تقدر تستخدم `Watson Language Translator API` — تبعتله النص ويأخذ منك اللغة المطلوبة، وهو يرجعك الترجمة جاهزة عن طريق `REST request` بصيغة `JSON`.

3. مجموعات البيانات في علم البيانات (Data Sets)

مجموعات البيانات هي تجميعات منظمة من المعلومات، بتشمل صيغ جدولية (`tabular`)، وهرمية (`hierarchical`)، وبيانات خام زي الصور والصوت.

مصادر البيانات: البيانات المفتوحة (Open Data) بتيجي من جهات عامة ومنصات زي Kaggle، اللي سهّلت الوصول للبيانات لأغراض البحث والتطوير.

الترخيص — Community Data License Agreement (CDLA)

نوع الترخيص	الشرط
CDLA-Sharing License	لو عدّلت في البيانات، لازم تشارك التعديلات ده كان (زي روح ال Copyleft في البرمجيات مفتوحة المصدر).
CDLA-Permissive License	تقدر تعدّل في البيانات من غير أي إلزام بمشاركة التعديلات.

4. مستودع IBM لأصول البيانات — IBM Data Asset eXchange (DAX)

نظرة عامة: DAX هو مستودع بيانات مفتوح من IBM، يوفر وصول لمجموعات بيانات عالية الجودة للتطبيقات على مستوى المؤسسات (enterprise-level).

- يوفر notebooks تعليمية ومتقدمة لمعالجة البيانات وتعلم الآلة والتحليل الإحصائي.
- يسمح للمستخدمين يستكشفوا ويحمّلوا ويدمجوا مجموعات البيانات في مشاريعهم زي Watson Studio.

مثال استخدام: DAX فيه مجموعات بيانات زي بيانات الطقس، اللي ممكن تُستكشف وتُحلل باستخدام الأدوات المتكاملة في IBM Watson Studio.

5. نماذج تعلم الآلة (Machine Learning Models)

أنواع التعلم

النوع	الوصف
Supervised Learning	يشمل ال regression (التنبؤ بقيم رقمية) وال classification (التنبؤ بفئات).
Unsupervised Learning	يشمل ال clustering (تجميع البيانات المتشابهة) وال anomaly detection (اكتشاف القيم الشاذة).
Reinforcement Learning	التعلم عن طريق المحاولة والخطأ بناءً على مكافآت (rewards).

التعلم العميق (Deep Learning)

- يحاكي عمل المخ البشري لحل مشاكل معقدة زي معالجة اللغة الطبيعية (NLP) وتحليل الصور.
- محتاج مجموعات بيانات ضخمة وهاردوير متخصص (GPU/TPU).

• أدوات زي TensorFlow وPyTorch وKeras تُستخدم لبناء وتدريب نماذج التعلم العميق.

6. أهم أدوات التعلم العميق (Key Tools for Deep Learning)

- الأطر البرمجية (Frameworks): TensorFlow وPyTorch وKeras هي الأطر الرائدة لبناء وتدريب النماذج.
- نماذج مدربة مسبقاً (Pre-Trained Models): يمكن نتاج من مستودعات زي TensorFlow Hub وPyTorch Hub لتطوير أسرع بدل التدريب من الصفر.
- خطوات بناء موديل مخصص (Custom Model Workflow): جمع البيانات → تجهيزها → تسميها (labeling) → اختيار أو بناء الموديل → التدريب → النشر.

خلاصة الوحدة الثانية (Key Takeaways)

- علم البيانات يعتمد بشكل كبير على مكتبات قوية (زي Pandas وScikit-learn) للتعامل مع البيانات والتصوير والتعلم الآلي.
- الـ APIs عنصر أساسي لدمج الخدمات المختلفة وتمكين التواصل بين البرمجيات.
- البيانات المفتوحة، بدعم من مبادرات زي DAX، بتلعب دور مهم في دعم البحث والابتكار في علم البيانات.
- تعلم الآلة والتعلم العميق هما العمود الفقري للتحليلات التنبؤية، وأطر زي TensorFlow بتقود التقدم في المجال.

إضافات من صور المحاضرة (Session 3)

1. مخطط فين — علاقة AI و ML و DL و Data Science

الصورة بتوضح دوائر متداخلة (Venn Diagram) بالترتيب التالي:

- **AI (Artificial Intelligence)**: أكبر دائرة، وهي المظلة العامة لأي نظام يحاكي الذكاء البشري.
- **ML (Machine Learning)**: دائرة جوه الـ AI، وهي فرع يتعلم من البيانات من غير برمجة صريحة لكل حالة.
- **DL (Deep Learning)**: دائرة جوه الـ ML، وهي فرع يستخدم الشبكات العصبية متعددة الطبقات.
- **Math & Statistics**: دائرة متقاطعة مع الكل، لأن أي نموذج تعلم آلة أو تعلم عميق مبني أساساً على أسس رياضية وإحصائية.
- **DS (Data Science)**: في المنتصف: توضيح إن علم البيانات هو نقطة التقاء كل ده — AI و ML و DL والرياضيات والإحصاء مع بعض.

مثال يوضح الفرق: لو عايز تتنبأ بسعر شقة بناءً على مساحتها وموقعها، ده Machine Learning تقليدي (regression). لو عايز تصنف صور قطط وكلاب من ملايين الصور، ده غالباً هيتحتاج Deep Learning (شبكة عصبية convolutional). و Data Science هو المجال الشامل اللي يجمع كل الأدوات والتقنيات دي مع فهم الإحصاء عشان توصل لقرار أو رؤية مفيدة من البيانات.

2. مفاهيم إضافية من الصور (Roboflow / CV / Repository)

Computer Vision (CV) — رؤية الحاسوب

التعريف: فرع من فروع الذكاء الاصطناعي بيهتم بتكوين الحاسوب من "فهم" وتفسير الصور والفيديوهات، زي ما العين والمخ البشري يفهموا المشهد اللي قدامهم.

أمثلة تطبيقية: التعرف على الوجوه (face recognition)، اكتشاف الأجسام في الصورة (object detection) زي التعرف على السيارات في كاميرات المرور، وقراءة النصوص من الصور (OCR).

Roboflow — أداة توسيم البيانات (Data Labeling)

التعريف: Roboflow منصة متخصصة في تجهيز بيانات الصور لمشاريع الـ Computer Vision. بتساعدك في رسم مربعات (bounding boxes) أو تحديد مناطق حول الأجسام في الصور (annotation)، وبعدين بتصدر الداتاسيت بصيغة جاهزة لتدريب موديلات زي YOLO أو أي موديل Object Detection تاني.

مثال عملي: عايز تدرب موديل يتعرف على "الخوذة" في صور عمال المصانع لأغراض السلامة المهنية. بتحمل الصور على Roboflow، ترسم مربع حوالين كل خوذة في كل صورة (توسيم/labeling)، والأداة بتساعدك كان تعمل augmentation (تدوير، تكبير، تغيير إضاءة) عشان تكبر حجم بيانات التدريب، وفي الآخر تصدّر الداتاسيت جاهز لتدريب موديل الكشف.

وده سبب ظهورها جنب مصطلح **Data Asset Management** في الصورة — لأنها فعلياً أداة لإدارة وتجهيز أصول بيانات الصور بالتحديد.

Repository — المستودع

التعريف: المستودع (Repository) هو المكان اللي بيتخزن فيه الكود أو المشروع كامل مع كل تاريخ التعديلات عليه (commit history). أشهر مثال عملي هو الـ "repo" اللي بتنشئه على GitHub.

مثال: مشروع تصنيف الصور بتاعك ممكن يكون عبارة عن Repository واحد على GitHub فيه: الكود (Python scripts / notebooks)، ملف requirements.txt للمكتبات، وملف README يشرح المشروع — وكل تعديل بتعمله بيتسجل كـ commit جديد تقدر ترجعه وقت ما تحب.

رابط Tasks بـ GitHub

الإشارة لـ "Tasks: GitHub" في الصورة بتوضح إن جزء من مهام المذاكرة أو التطبيق العملي في المحاضرة مرتبط باستخدام GitHub — يعني غالباً فيه تكليف عملي (assignment) لازم تسلمه أو تطبقه من خلال إنشاء Repository ورفع الكود عليه.

3. جدول مقارنة شامل — بيئات التطوير والتنفيذ

المفهوم	أمثلة	الاستخدام الأساسي
Development Environment	Jupyter, Colab, Anaconda, VS Code, PyCharm	كتابة وتجربة الكود محلياً أو سحابياً
Execution Environment	AWS, IBM Cloud, Google Cloud, Apache Spark	تشغيل الكود بقوة حوسبة أعلى (خصوصاً للبيانات الضخمة)
Code Asset Management	Git, GitHub, Repository	تتبع نسخ الكود والتعاون الجماعي
Data Asset Management	DVC, MLflow, Git LFS, Roboflow (للصور)	تتبع نسخ البيانات وتوسيمها وتجهيزها
Fully Integrated Tools	IBM Watson Studio, Azure ML, Cognos Analytics	تجميع كل مراحل المشروع في مكان واحد

أسئلة مراجعة سريعة (Quick Review)

س1: إيه الفرق الأساسي بين Data Asset Management و Code Asset Management؟

س2: إيه Google Colab مفيد جداً في مشاريع Deep Learning عن باقي بيئات التطوير؟

س3: إيه الفرق بين IBM Watson Studio و IBM Cognos Analytics؟

س4: اذكر نوعين من تراخيص CDLA ووضح الفرق بينهم.

س5: إيه دور Roboflow في مشاريع Computer Vision، وليه هو Data Asset Management tool وليس Development Environment؟

س6: رتب دوائر مخطط فين (Venn Diagram) من الأكبر للأصغر: AI, ML, DL.

نصائح سريعة للمذاكرة

- ركّز على الجداول المقارنة (بيئات التطوير، الفئات الخمس للأدوات) لأنها الأكثر ترجيحاً في أسئلة الاختيار من متعدد.
- افهم السبب وراء كل تصنيف مش بس احفظ الاسم — مثلاً ليه Roboflow اتخطت جنب Data Asset Management مش Development Environments.
- اربط كل أداة بمراحلها في دورة حياة المشروع: جمع → تنظيف → بناء → نشر → مراقبة.